Project Report: AI Agent for Asalto Board Game

1. Introduction

According to the general guidance and requirements, our teams start to develop an Artificial Intelligence agent capable of playing the traditional board game Asalto. 'Asalto' is an asymmetric game, which means both sides(Rebels and Officers) have totally different goals, moving rules as well as the amount of chess piece. In the game, rebels rely on swarming tactics to occupy the fortress, while two officers need to capture pieces to survive. The aim of our project is to build a balance, robust game bot, which can handle both roles effectively under the strict 10-second time limit imposed by the tournament rules.

For the implementing, **we use a Minimax algorithm, enhanced Alpha-Beta pruning and Iterative Deepening**, because we thought this might be the most reliable approach for this deterministic, perfect-information game. Whereas, we extended beyond traditional search algorithms. During the designing and coding stage, we also use the 'Deep Reinforcement Learning (DQN)' to find a better performance. Although we finally selected the engine based on search due to it's stability in handling mandatory capture rules. But the experience we gained from DQN and training the neural network played a vital role in improving our heuristic evaluation functions. This report details our theoretical choices, the mathematical design of our heuristics, our implementation strategy, and the lessons learned from our experimental failures.

2. Theoretical Basis

2.1 Search Strategy: Minimax with Iterative Deepening

Asalto is a zero-sum game, so we choose Minimax algorithm as our foundational strategy. The core concepts of this algorithm are simulating future moves to minimize the maximum possible loss, assuming the opponent plays optimally. In the design, we let s be the current state and v(s) be the value of the state. The algorithm computes the optimal move by solving the recursive equation:

$$v(s) = \begin{cases} \max\limits_{a \in Actions(s)} v(result(s,a)) & if\ Player = Max \\ \min\limits_{a \in Actions(s)} v(result(s,a)) & if\ Player = Min \end{cases} \quad (1)$$

However, the game tree for Asalto can be very deep. In order to make real-time decisions useful, we have used 'Alpha-Beta pruning'. This optimization method allows the algorithm to "cut off" branches of the search tree, this branches might have worse performance to what we already found. This will guarantee the efficiency of doubling our search depth in the best case.

For the strategy of our project, a significant addition is Iterative Deepening. In the early development, we faced a problem: specifying a fixed search depth (e.g., depth 5) was risky. If the branching factor exploded—common when playing as Rebels—the bot would timeout and crash. After using Iterative Deepening, our agent searches for depth 1, then depth 2, and so on, incrementally. We check the system clock after every iteration. If the time approaches the 9-second safety buffer, we immediately halt the calculation and return the best move found in the previous fully completed depth. This transforms our algorithm into an "Anytime Algorithm," ensuring the bot always has a valid and safe move ready to play. No matter how complex the board state becomes.

2.2 Heuristic Evaluation Functions: The Mathematics of Strategy

When we searching the game tree to the final "checkmate" is impossible within 10 seconds, we found the quality of our bot depends entirely on its Heuristic Evaluation Function (evaluate_board). So we adjust weights by a specific mathematical to encode strategic behaviors(as below):

Officer (Defensive Strategy): The Officer's evaluation function is unfair and heavily weighted toward material advantage. We express the heuristic function $H_{Officer}(s)$ as a linear combination of the following strategic factors: For the Officer (Defensive Strategy)

$$H_{Officer}(s) = w_{cap} \cdot (24 - N_{rebel}) + w_{def} \cdot I_{fort} + w_{mob} \cdot M_{officer} - w_{cent} \cdot D_{center} \quad (2)$$

$\boldsymbol{w_{cap} = 500}$**:** The huge weights of forced capture ensure that the capture rules are always given priority.

$\boldsymbol{w_{def} = 100}$ **:** Bonus for occupying the critical defense points $(2,2), (2,3), (2,4), (I_{fort})$.

$\boldsymbol{w_{mob} = 20}$**:** Reward for $Mobility(M_{officer})$, calculated as the number of available moves to prevent entrapment.

$\boldsymbol{D_{center}}$**:** Penalty based on the distance from the center, encouraging Officers to stay active in the middle.

Rebel (Swarm Strategy): The Rebel logic was harder to tune because they are individually weak. To quantify their positional advantage, we defined the Rebel heuristic $H_{Rebel}(s)$ as follows:

$$H_{Rebel}(s) = \sum_{r \in Rebels}(w_{in} \cdot P(r) - w_{dist} \cdot |r - target| + w_{clus} \cdot N_{neighbors}(r)) \quad (3)$$

$(\boldsymbol{w_{in} \cdot P(r)})$**:** We used a dynamic score $P(r) = 200 + (2 - row) \times 20$ to reward Rebels not just for entering the fortress, but for pushing to the back rank to avoid clogging the entrance.

$(\boldsymbol{w_{clus} \cdot N_{neighbors}})$**:** Single Rebels are easily captured. We added a bonus for each adjacent teammate $N_{neighbors}$, effectively mathematically encouraging a "Wolf Pack" formation that is harder to penetrate.

2.3 Exploration method: Deep Q-Network (DQN) Architecture

Although not used in the final tournament, we designed a custom Convolutional Neural Network (CNN) to learn the game. It is a 3-channel 7×7 CNN (3 conv + 2 FC layers, Tanh output) is trained end-to-end to minimize MSE against Bellman targets for Q-values. During the training phase, we aimed to minimize the Mean Squared Error (MSE) loss between the predicted Q-values and the target values derived from the Bellman equation:

$$L(\theta) = E[(r + \gamma \max_{a'} Q(s', a'; \theta^{-}) - Q(s, a; \theta))^2] \quad (4)$$

The reason why we give up this method is due to three attempts——First, we try single DQN100%minimax guidance for learning matches, it shows a poor effect. Second, we try double DQN100%minimax guidance for matches, it still poor effect plus more time cost. The last try, we use double DQN+MCTS. The ‘officers’ uses minimax, the ‘rebel’ uses DQN+MCTS. The training process starts with 2000 rounds of observing minimax matches (imitation). After that, started to practice and play against minimax, the effect was not that good and slow. This failure reinforced our decision to stick with the rule-based Minimax for the competition.

3. Implementation Details

We implemented the project using Python 3, strictly consistent with the requirements.txt to ensure compatibility with the testing framework. We avoided external libraries like pandas or scikit-learn in the final submission to prevent environment errors.

Class Structure and Modularity: We structured our code to separate concerns clearly. The main Player class in Team20.py acts as a lightweight controller. The actual logic depends on two specialized modules: OfficerAI and RebelAI. This separation allowed different team members to work on the Rebel and Officer logic simultaneously without merge conflicts.

State Representation: We used a standard 2D list to represent the board. While bitboards (using 64-bit integers) would be faster, the 2D list approach allowed for directly debugging and easier implementation of the CNN input tensor during our research phase.

Move Generation: We implemented separate move generators for each role. The get_all_rebel_moves function is particularly optimized to filter out backward moves immediately, reducing the branching factor before the search even begins.

Robust Time Management A critical technical feature is our exception-based time control. We did not rely on simply checking the time before a move. Instead, we embedded a time check deep inside the recursive minimax function.

```
if start_time and (time.time() - start_time > TIME_LIMIT):
    raise TimeoutError
```

By raising a custom exception, we can start the recursion stack from any depth instantly. This was handled by a try… except block in the main loop, which catches the error and returns the result from the previous depth. This will guarantee that we maximize the usage of the 10-second window without ever exceeding it.

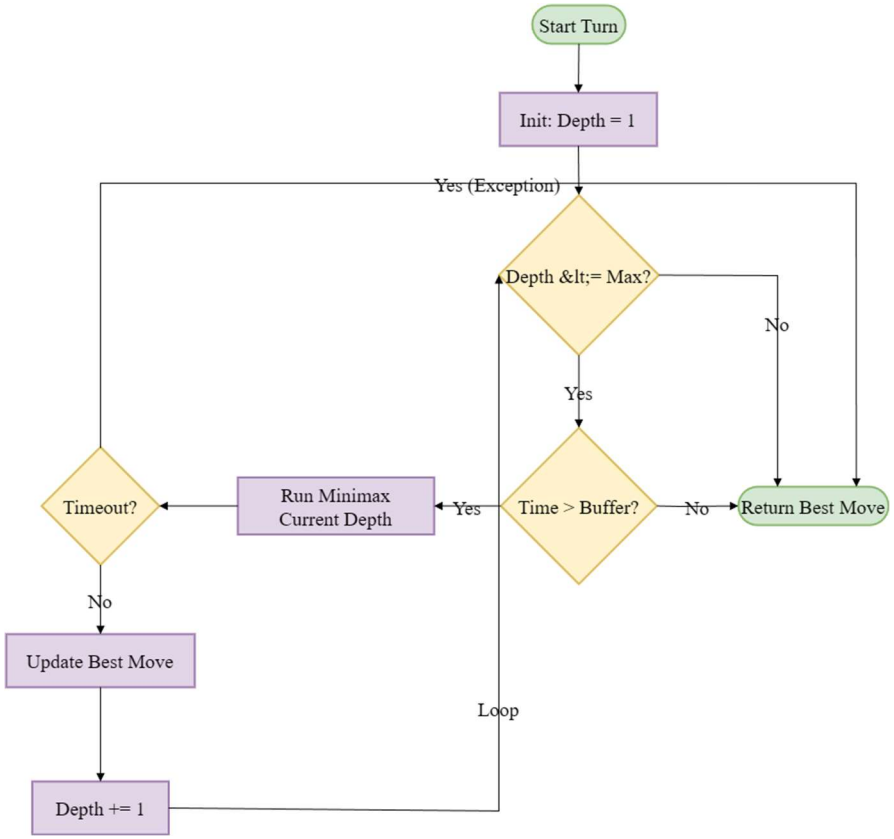

Figure 1 : A flowchart showing the Iterative Deepening Minimax process with Exception Handling

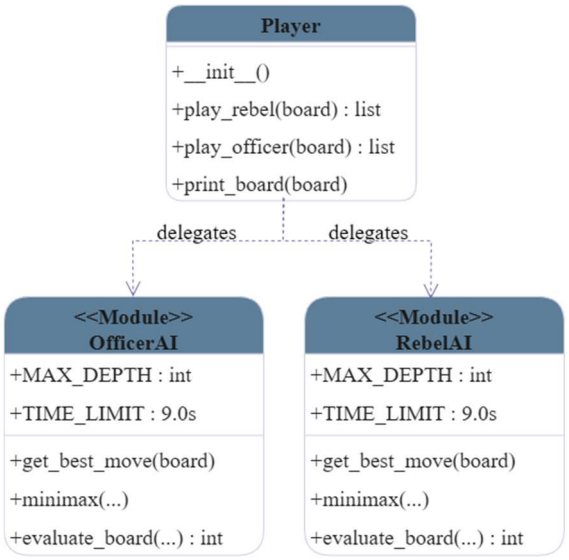

Figure 2 : A UML Class Diagram showing the relationship between Player, OfficerAI, and RebelAI classes

4. Conclusions

4.1 Reflections on Performance

We have met a unique challenge because the asymmetry of Asalto. Under the observation of us, we found the Branching Factor for Rebels is significantly higher (often 20+ moves) than for Officers (often <5 moves). Which means our Officer AI can consistently search deeper (Depth 6-7) than our Rebel AI (Depth 4-5) within the same time limit. To solve this, we put more effort on heuristic function for Rebel's shallower search, especially the "Manhattan Distance" guidance. It can act as a compass to guide the search in the right direction even at shallow depths.

4.2 Difficulties Faced

The "Horizon Effect": We have faced a situation where the Officer would delay a loss rather than preventing it or move back and forth between two safe squares (Deadlock). We mitigated this through introducing a random shuffle to equivalent moves, which can ensure the bot explores different paths rather than enter an infinite loop.

Mandatory Capture Constraints: Initially, our Officer AI would sometimes miss a capture because it considers positional advantage is more valuable. We fixed this by assigning the capture weight (500) to be larger than the maximum possible sum of all other positional weights combined, making capturing the mathematical imperative.

4.3 Future Improvements

Transposition Tables: Currently, our bot will recalculates the value of the same board if it encounters it again. Hence, we plan to implement a Zobrist Hash table, which would allow us to cache these results, potentially improving our search speed.

Hybrid Search: A "Minimax-Guided MCTS" approach could be powerful. Instead of random rollouts in Monte Carlo Tree Search, we could use our shallow Minimax evaluation to guide the simulation, combining the strategic foresight of Minimax with the statistical robustness of MCTS.

Bitboard Optimization: We have meet a limitation, which is rewriting the board representation using bitwise operations would significantly speed up the `is_valid_move` checks, which are currently hard to solve in our Python implementation.

5. References and Acknowledgements

Russell, S., & Norvig, P. *Artificial Intelligence: A Modern Approach*. (Theoretical basis for Minimax and Alpha-Beta pruning).

DeepMind. *Human-level control through deep reinforcement learning*. (Inspiration for the CNN architecture in TeamDQN.py).

Asalto Game Rules. Cyningstan. (Used to verify move validity and capture mechanics).